LangGraph Mastery Guide

Phase 2: Control Flow — Conditional Edges, Cycles & The Command/Send API

TypeScript · Node.js · OpenAI

Who this guide is for: Someone who has completed Phase 1 and can build a basic StateGraph with nodes and edges. You understand `invoke()`, `compile()`, and how state flows between nodes. If any of that sounds unfamiliar, revisit Phase 1 before continuing here.

What you will be able to do after this guide:

- Route graph execution to different nodes based on runtime conditions
- Build self-correcting loops that repeat until a quality threshold is met
- Safely prevent infinite loops with recursion limits
- Use the `Command` object to make routing decisions from inside a node
- Fan out work to multiple parallel nodes using the `Send` API
- Build a map-reduce pipeline that processes data in parallel and aggregates results

---

Table of Contents

Project Setup

Before writing any code, get your environment ready. Do this once at the start of the phase.

Step 1 — Create the project folder

```
mkdir langgraph-phase2
cd langgraph-phase2
npm init -y
```

Why: `npm init -y` creates a `package.json` with default values. This file tracks your project's dependencies and scripts.

Step 2 — Install dependencies

```
npm install @langchain/langgraph @langchain/openai @langchain/core dotenv
npm install -D typescript tsx @types/node
```

What each package does:

Package	Purpose
`@langchain/langgraph`	The core library — StateGraph, nodes, edges, Command, Send
`@langchain/openai`	LangChain's wrapper for OpenAI's API (ChatOpenAI)
`@langchain/core`	Shared LangChain types — BaseMessage, HumanMessage, etc.
`dotenv`	Loads your `.env` file so your API key is accessible
`typescript`	The TypeScript compiler
`tsx`	Runs TypeScript files directly without compiling first (like `ts-node` but faster)
`@types/node`	TypeScript type definitions for Node.js built-ins

Step 3 — Create tsconfig.json

Create a file called `tsconfig.json` in the root of your project:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "outDir": "./dist"
  },
  "include": ["src/**/*"]
}
```

Why: This tells TypeScript how to compile your code. `strict: true` enables all safety checks. `ES2022` supports modern JS features like top-level `await`.

Step 4 — Create your .env file

```
touch .env
```

Open it and add:

```
OPENAI_API_KEY=sk-your-actual-key-here
```

> **Important:** Never commit your `.env` file to Git. Create a `.gitignore` and add `.env` to it.

Step 5 — Create the folder structure

```
mkdir -p src/unit3 src/unit4 src/final-project
```

Your project should look like this:

```
langgraph-phase2/
├── src/
│   ├── unit3/
│   │   ├── lab1.ts
│   │   ├── lab2.ts
│   │   ├── lab3.ts
│   │   └── lab4.ts
│   ├── unit4/
│   │   ├── lab1.ts
│   │   ├── lab2.ts
│   │   ├── lab3.ts
│   │   └── lab4.ts
│   └── final-project/
│       └── pipeline.ts
├── .env
├── tsconfig.json
└── package.json
```

Step 6 — Add a run script to package.json

Open `package.json` and add a `scripts` section:

```
{
  "scripts": {
    "run:lab": "tsx"
  }
}
```

How to run any lab after this:

```
npx tsx src/unit3/lab1.ts
```

UNIT 3: Conditional Edges & Branching

Chapter 1: Why Static Edges Are Not Enough

1.1 A Recap of Static Edges

In Phase 1, you learned that `addEdge("nodeA", "nodeB")` creates a permanent, unconditional connection. Every time `nodeA` finishes, execution **always** moves to `nodeB`. No exceptions.

This is fine for linear pipelines — think of an assembly line where every step always follows the previous one in order.

`START → fetchData → processData → saveData → END`

But real-world logic is rarely that simple.

1.2 The Problem with Always Going the Same Way

Imagine you are building a customer support bot:

- If the user asks a billing question → route to the billing agent
- If the user asks a technical question → route to the tech support agent
- If the query is unclear → route to a clarification node

With static edges, you **cannot** do this. A static edge is a one-way street with no traffic light. You need a traffic light — something that looks at the current situation and decides which road to take.

That traffic light is a **conditional edge**.

1.3 What a Conditional Edge Actually Is

A conditional edge is a special connection that, instead of always pointing to one fixed node, calls a **routing function** first. The routing function looks at the current graph state and returns the name of the next node to go to.

Analogy — The Airport Departure Board

Think of a conditional edge as an airport departure board. When you arrive at the gate check, a staff member (the routing function) looks at your boarding pass (the state), and directs you to:

- Gate A12 (if flying domestic)
- Gate B7 (if flying international)
- The check-in desk again (if something is missing)

The routing function is that staff member. It inspects the state and makes a decision.

1.4 When Do You Use Conditional Edges?

Use conditional edges when:

- Different paths through your graph should be taken based on runtime data
- You need to loop back to an earlier node until a condition is met
- You want to skip certain nodes based on user input or LLM output
- You are implementing approval flows (e.g., ”if confidence > 0.9, auto-approve; otherwise, request review”)

Chapter 2: addConditionalEdges() — The Full Picture

2.1 The Function Signature

```
graph.addConditionalEdges(
  sourceNode: string,
  routingFn: (state: StateType) => string | string[],
  pathMap?: Record<string, string>   // optional
);
```

There are three parts:

1. **`sourceNode`** — The name of the node *after which* the routing decision happens. When this node finishes, instead of going to a fixed next node, LangGraph calls your routing function.
2. **`routingFn`** — A plain function that receives the current state and returns either:
   - A single string: the name of the next node to go to
   - An array of strings: multiple node names (for parallel fan-out — see Chapter 3)
3. **`pathMap`** (optional) — A dictionary that maps symbolic return values from your routing function to actual node names. This is useful for readability and for routing to the special `END` constant.

2.2 A Minimal Example — No Path Map

```
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";

// Define state
const ReviewState = Annotation.Root({
  score: Annotation<number>({
    reducer: (_, b) => b,
    default: () => 0
  })
});

// Define nodes
function evaluate(state: typeof ReviewState.State) {
  return { score: 85 }; // pretend we computed a score
}

function approve(state: typeof ReviewState.State) {
  console.log("Approved! Score was:", state.score);
  return {};
}

function reject(state: typeof ReviewState.State) {
  console.log("Rejected. Score was:", state.score);
  return {};
}

// The routing function — this is the traffic light
```

```
function routeByScore(state: typeof ReviewState.State): string {
  if (state.score >= 80) {
    return "approve";  // return the NODE NAME to go to next
  }
  return "reject";
}

// Build the graph
const graph = new StateGraph(ReviewState)
  .addNode("evaluate", evaluate)
  .addNode("approve", approve)
  .addNode("reject", reject)
  .addEdge(START, "evaluate")
  .addConditionalEdges("evaluate", routeByScore)  // after "evaluate", call routeByScore
  .addEdge("approve", END)
  .addEdge("reject", END);

const compiled = graph.compile();

const result = await compiled.invoke({});
```

What happens step by step:

1. Graph starts at `START`, moves to `evaluate` (static edge)
2. `evaluate` runs, sets `score: 85` in state
3. LangGraph sees a conditional edge after `evaluate`, calls `routeByScore`
4. `routeByScore` reads `state.score` (85), returns `"approve"`
5. LangGraph moves execution to the `approve` node
6. `approve` runs, then moves to `END` (static edge)

2.3 Using a Path Map (Optional Third Argument)

Sometimes you want your routing function to return short, readable values (`"yes"`, `"no"`) rather than actual node names. The path map translates those values to real node names:

```
function routeByScore(state: typeof ReviewState.State): string {
  return state.score >= 80 ? "pass" : "fail";  // symbolic values
}

graph.addConditionalEdges(
  "evaluate",
  routeByScore,
  {
    pass: "approve",   // "pass" → go to "approve" node
    fail: "reject"     // "fail" → go to "reject" node
  }
);
```

Why use a path map?

- Your routing function reads like English: `return "pass"` is cleaner than `return "approveDocumentAndNotifyUser"`
- It is the only way to route to the `END` constant without importing it into your routing function:

```
graph.addConditionalEdges(
  "checkNode",
  routeFn,
  {
    done: END,      // route to the END constant
    retry: "fixNode"
  }
);
```

2.4 Type Safety for Routing Functions

In TypeScript, it is good practice to type your routing function's return value:

```
type RouteResult = "approve" | "reject";

function routeByScore(state: typeof ReviewState.State): RouteResult {
  return state.score >= 80 ? "approve" : "reject";
}
```

This way TypeScript will warn you if you accidentally return a node name that does not exist.

Chapter 3: Fan-Out — One Source, Many Possible Destinations

3.1 What "Fan-Out" Means

Fan-out means that from a single source node, execution can go to **multiple different possible destinations** — but only ONE of them is chosen at runtime based on the state.

This is different from going to multiple nodes in parallel (that is what `Send` does in Unit 4). Fan-out means you have 3 possible roads, but you pick exactly 1.

Analogy — A Hotel Receptionist

A hotel receptionist can direct a guest to:

- Room 101 (if the guest booked a standard room)
- Room 201 (if the guest booked a deluxe room)
- Room 301 (if the guest booked a suite)
- The manager's office (if there is a problem)

Only one of those happens per guest. That is fan-out.

3.2 Fan-Out in Code

Fan-out is just `addConditionalEdges` with more than 2 possible return values:

```
function routeSupport(state: typeof SupportState.State): string {
  const intent = state.intent;

  if (intent === "billing") return "billingAgent";
  if (intent === "technical") return "techAgent";
  if (intent === "complaint") return "complaintAgent";
  return "generalAgent"; // fallback
}
```

```
graph.addConditionalEdges("classifyIntent", routeSupport);
// Now "classifyIntent" can fan out to any of 4 nodes
```

3.3 The Importance of a Fallback Path

Always include a fallback return value in your routing function. If your routing function ever returns a string that is not a valid node name and you have not handled it, LangGraph will throw an error.

```
// Bad — no fallback
function routeFn(state) {
  if (state.x === "a") return "nodeA";
  if (state.x === "b") return "nodeB";
  // what if state.x is "c"? This crashes.
}

// Good — always has a fallback
function routeFn(state) {
  if (state.x === "a") return "nodeA";
  if (state.x === "b") return "nodeB";
  return "fallbackNode"; // always returns something valid
}
```

Chapter 4: Cycles and Loops — LangGraph's Killer Feature

4.1 Why Cycles Change Everything

This is the most important concept in Phase 2. It is the core reason LangGraph exists.

In a standard LangChain chain (Phase 0), execution is **linear** — it goes forward and never repeats a step. In most real-world AI applications, this is too rigid.

Think about how a human expert writes a report:

1. Write a draft
2. Review the draft
3. If the draft is good enough, finish. If not, go back to step 1 and rewrite.

This is a **cycle** — step 3 can point back to step 1. Standard chains cannot do this. LangGraph can.

Analogy — A Kitchen Pass

In a restaurant kitchen, a plate goes from the chef to the pass (the inspection counter). The head chef inspects it:

- If it is perfect → it goes out to the customer
- If it is not → it goes back to the chef for revision

That loop between chef → pass → chef is a cycle. LangGraph lets you model this directly in your graph.

4.2 How to Create a Cycle

A cycle is created when your routing function sends execution to a node that already ran (i.e., an earlier node in the graph). There is no special `addCycle()` function — you just route

backwards.

```
START → generate → critique → [conditional] → generate (again if bad)
                                             → END (if good)
```

In code:

```typescript
const WriterState = Annotation.Root({
  draft: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  feedback: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  iterations: Annotation<number>({ reducer: (_, b) => b, default: () => 0 }),
  approved: Annotation<boolean>({ reducer: (_, b) => b, default: () => false })
});

function generate(state: typeof WriterState.State) {
  // Generate or revise a draft
  return { draft: "some draft text", iterations: state.iterations + 1 };
}

function critique(state: typeof WriterState.State) {
  // Evaluate the draft
  const isGood = state.draft.length > 100; // simplified check
  return { approved: isGood, feedback: isGood ? "" : "Too short, expand it." };
}

function shouldContinue(state: typeof WriterState.State): string {
  if (state.approved) return END;          // exit the loop
  if (state.iterations >= 3) return END;   // safety limit
  return "generate";                       // LOOP BACK to generate
}

const graph = new StateGraph(WriterState)
  .addNode("generate", generate)
  .addNode("critique", critique)
  .addEdge(START, "generate")
  .addEdge("generate", "critique")
  .addConditionalEdges("critique", shouldContinue);
  //                                  ↑ can return "generate" (loop back) or END
```

4.3 The Three-Part Loop Pattern

Every loop in LangGraph follows this structure:

```
[WORK NODE] → [JUDGE NODE] → [CONDITIONAL EDGE]
                                  ↓ "not done yet" → back to [WORK NODE]
                                  ↓ "done" → END (or next step)
```

- **Work node:** Does the actual task (generate text, call an API, etc.)
- **Judge node:** Evaluates whether the work meets the criteria
- **Conditional edge:** Routes back to the work node if criteria are not met, or forwards if they are

4.4 Termination Conditions — Always Have an Exit

A loop without an exit runs forever. Always include at least **two** exit conditions in every loop:

1. **Success condition:** ”The work is good enough — move on.”
2. **Iteration cap:** ”We have tried N times — move on regardless.”

```ts
function shouldContinue(state: typeof WriterState.State): string {
  // Exit condition 1 — success
  if (state.approved) return END;

  // Exit condition 2 — iteration cap (safety net)
  if (state.iterations >= 5) {
    console.warn("Max iterations reached. Exiting with last draft.");
    return END;
  }

  // Continue looping
  return "generate";
}
```

Chapter 5: recursionLimit — Your Safety Net

5.1 What is recursionLimit?

Even with a good iteration cap inside your routing function, things can go wrong. `recursionLimit` is LangGraph's built-in hard stop. It is a number you pass at runtime when calling `invoke()` or `stream()`. If the total number of node executions exceeds this number, LangGraph throws an error and stops execution.

Think of it as the circuit breaker in your home's electrical panel. Your home has multiple safety mechanisms, but if everything else fails, the circuit breaker trips and cuts the power before a fire starts.

5.2 Default Value

The default `recursionLimit` is **25 steps**. This means your graph can execute at most 25 node runs total before LangGraph automatically terminates with an error.

5.3 How to Set It

You pass `recursionLimit` as part of the config object in the second argument to `invoke()`:

```ts
const result = await compiled.invoke(
  { input: "write me an essay" },
  { recursionLimit: 10 }  // stop after 10 node executions total
);
```

5.4 When to Increase or Decrease It

Situation	Recommended Action
Simple graph with no loops	Leave at default (25) — it will never be hit
Graph with a loop that runs 3–5 times max	Set to `20–30`
Graph with many parallel nodes (Unit 4)	Increase it — each parallel execution counts toward the limit
Production system where you need strict control	Set it low (`10–15`) to catch runaway loops early
Development/testing	Set higher temporarily to debug without interruption

5.5 The Error You Will See

When `recursionLimit` is exceeded, LangGraph throws an error message containing `GraphRecursionError` or similar. You can catch it like any JavaScript error:

```
try {
  const result = await compiled.invoke(input, { recursionLimit: 5 });
} catch (error) {
  if (error instanceof Error && error.message.includes("recursion")) {
    console.error("Graph exceeded recursion limit:", error.message);
  } else {
    throw error;
  }
}
```

Practical Labs — Unit 3

Now you will apply everything from the chapters above. Each lab is a complete, runnable TypeScript file.

Lab 3.1: Content Moderation Router

What you will build: A graph that takes a user message, classifies it as either `safe` or `flagged`, and routes it to the appropriate handler node.

What you will learn:

- Writing a routing function that reads LLM output
- Fan-out with 2 possible destinations
- How to use OpenAI for classification inside a node

Why This Matters

Content moderation is a real-world use case. Before sending a message to a downstream system, you often need to check it first. This graph models that gate.

Step 1 — Create the file

```
touch src/unit3/lab1.ts
```

Step 2 — Write the code

```
// src/unit3/lab1.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// ──────────────────────────────────────────
```

```
// 1. Define State
// ─────────────────────────────────────────
// This is the shared "memory" of our graph.
// Every node can read from it and write partial updates to it.

const ModerationState = Annotation.Root({
  // The raw user message we want to moderate
  userMessage: Annotation<string>({
    reducer: (_, b) => b,   // last-write-wins: new value always replaces old
    default: () => ""
  }),
  // The classification result: "safe" or "flagged"
  classification: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  }),
  // The final response we send back to the user
  response: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  })
});

// Create the LLM — we use gpt-4o-mini for cost efficiency
const llm = new ChatOpenAI({
  model: "gpt-4o-mini",
  temperature: 0  // temperature 0 = deterministic, no creativity (good for
 ↪  classification)
});

// ─────────────────────────────────────────
// 2. Define Nodes
// ─────────────────────────────────────────

// NODE 1: Classify the message
// Why: We need to know whether to handle this message or block it.
// This node calls the LLM and asks it to classify the message.
async function classifyMessage(
  state: typeof ModerationState.State
) {
  console.log("\n[Node: classifyMessage] Analyzing:", state.userMessage);

  const response = await llm.invoke([
    new SystemMessage(
      `You are a content moderation system.
       Classify the user message as either "safe" or "flagged".
       Respond with ONLY the word "safe" or "flagged" — nothing else.`
    ),
    new HumanMessage(state.userMessage)
  ]);

  const result = (response.content as string).trim().toLowerCase();
  console.log("[Node: classifyMessage] Classification:", result);

  // Return only the fields we are updating
  return { classification: result };
}
```

```
// NODE 2: Handle safe messages
// Why: Safe messages get a normal, helpful response.
async function handleSafe(
  state: typeof ModerationState.State
) {
  console.log("[Node: handleSafe] Processing safe message...");

  const response = await llm.invoke([
    new SystemMessage("You are a helpful assistant. Respond to the user's message."),
    new HumanMessage(state.userMessage)
  ]);

  return { response: response.content as string };
}

// NODE 3: Handle flagged messages
// Why: Flagged messages should not be processed — we explain why we cannot help.
async function handleFlagged(
  state: typeof ModerationState.State
) {
  console.log("[Node: handleFlagged] Message was flagged!");
  return {
    response:
      "I'm sorry, I'm unable to process that message as it appears to violate our content
       ↪  policy."
  };
}

// ─────────────────────────────────────────────
// 3. Define the Routing Function
// ─────────────────────────────────────────────

// This is the traffic light. It reads state.classification
// and returns the NAME of the next node.
function routeByClassification(
  state: typeof ModerationState.State
): string {
  console.log("[Router] Routing based on classification:", state.classification);

  if (state.classification === "safe") {
    return "handleSafe";    // go to the safe handler node
  }
  return "handleFlagged";   // go to the flagged handler node (default/fallback)
}

// ─────────────────────────────────────────────
// 4. Build the Graph
// ─────────────────────────────────────────────
const graph = new StateGraph(ModerationState)
  // Register all nodes
  .addNode("classifyMessage", classifyMessage)
  .addNode("handleSafe", handleSafe)
  .addNode("handleFlagged", handleFlagged)

  // Static edges: where to go unconditionally
  .addEdge(START, "classifyMessage")           // always start with classification
```

```
  .addEdge("handleSafe", END)                 // safe path ends here
  .addEdge("handleFlagged", END)              // flagged path ends here

  // Conditional edge: after classifyMessage, call routeByClassification
  .addConditionalEdges("classifyMessage", routeByClassification);

const compiled = graph.compile();

// ─────────────────────────────────────────────
// 5. Run the Graph — Test Both Paths
// ─────────────────────────────────────────────
async function main() {
  console.log("=== TEST 1: Safe Message ===");
  const result1 = await compiled.invoke({
    userMessage: "What is the capital of France?"
  });
  console.log("Response:", result1.response);

  console.log("\n=== TEST 2: Flagged Message ===");
  const result2 = await compiled.invoke({
    userMessage: "How do I hack into someone's email account?"
  });
  console.log("Response:", result2.response);
}

main().catch(console.error);
```

Step 3 — Run it

```
npx tsx src/unit3/lab1.ts
```

Expected Output

```
=== TEST 1: Safe Message ===
[Node: classifyMessage] Analyzing: What is the capital of France?
[Node: classifyMessage] Classification: safe
[Router] Routing based on classification: safe
[Node: handleSafe] Processing safe message...
Response: The capital of France is Paris.

=== TEST 2: Flagged Message ===
[Node: classifyMessage] Analyzing: How do I hack into someone's email account?
[Node: classifyMessage] Classification: flagged
[Router] Routing based on classification: flagged
[Node: handleFlagged] Message was flagged!
Response: I'm sorry, I'm unable to process that message...
```

How to Verify It Worked

- Test 1 should go through `handleSafe` and return a real answer.
- Test 2 should go through `handleFlagged` and return a refusal message.
- The console logs show you exactly which node ran and in what order.

Common Mistakes

Mistake	Symptom	Fix
Routing function returns a node name that does not exist	Runtime error: "Node not found"	Double-check your r
Forgot to add `addEdge(handleSafe, END)`	Graph hangs after handleSafe	Every terminal node
LLM returns "Safe" with capital S	Router falls through to flagged path	Always call `.toLowe`
Missing `dotenv/config` import	`OPENAI_API_KEY is undefined`	Add `import "dotenv`

Lab 3.2: AI Essay Writer with Self-Correction Loop

What you will build: A graph that generates an essay, critiques it, and loops back to rewrite it until the critique approves it or 3 iterations are reached.

What you will learn:

- Building a cycle (loop) in a graph
- Using iteration counters for safety
- Passing feedback between loop iterations via state

Why This Matters

This is the most common pattern in production AI systems — "agentic loops." Instead of running the LLM once and accepting the result, you run it multiple times until the output meets your standard. This is called a self-correcting or reflection loop.

Create and write the file

```
// src/unit3/lab2.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// ─────────────────────────────────────────
// 1. State
// ─────────────────────────────────────────
const EssayState = Annotation.Root({
  topic: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  }),
  draft: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  }),
  feedback: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  }),
  // approved: did the critique approve the draft?
  approved: Annotation<boolean>({
    reducer: (_, b) => b,
```

```
    default: () => false
  }),
  // iterations: how many times have we generated?
  iterations: Annotation<number>({
    reducer: (_, b) => b,
    default: () => 0
  })
});

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.7 });
const judger = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

// ─────────────────────────────────────────
// 2. Nodes
// ─────────────────────────────────────────

// NODE 1: Generate or revise the essay
// On first run: writes from scratch
// On subsequent runs: incorporates the feedback from the critique node
async function generate(state: typeof EssayState.State) {
  console.log(`\n[Node: generate] Iteration ${state.iterations + 1}`);

  const isRevision = state.iterations > 0;

  const messages = isRevision
    ? [
        new SystemMessage(
          "You are an essay writer. Revise the essay based on the feedback provided. " +
          "Keep the essay between 150 and 250 words."
        ),
        new HumanMessage(
          `Topic: ${state.topic}\n\nCurrent
          ↪  draft:\n${state.draft}\n\nFeedback:\n${state.feedback}\n\nPlease revise
          ↪  the essay.`
        )
      ]
    : [
        new SystemMessage(
          "You are an essay writer. Write a short essay between 150 and 250 words."
        ),
        new HumanMessage(`Write an essay on: ${state.topic}`)
      ];

  const response = await llm.invoke(messages);
  const newDraft = response.content as string;
  console.log("[Node: generate] Draft length:", newDraft.split(" ").length, "words");

  return {
    draft: newDraft,
    iterations: state.iterations + 1  // increment the counter
  };
}

// NODE 2: Critique the draft
// Decides if the draft is good enough.
// Returns approved: true if it passes, plus feedback if it does not.
async function critique(state: typeof EssayState.State) {
```

```
  console.log("[Node: critique] Evaluating draft...");

  const response = await judger.invoke([
    new SystemMessage(
      `You are a strict essay critic.
       Evaluate the essay on these criteria:
       1. Is it between 150 and 250 words?
       2. Does it have a clear introduction, body, and conclusion?
       3. Is the writing coherent and on-topic?

       Respond in this exact format:
       VERDICT: APPROVED or REJECTED
       FEEDBACK: (if rejected, explain what needs to change in one sentence)`
    ),
    new HumanMessage(`Topic: ${state.topic}\n\nEssay:\n${state.draft}`)
  ]);

  const raw = response.content as string;
  console.log("[Node: critique] Raw critique:\n", raw);

  const approved = raw.includes("VERDICT: APPROVED");
  const feedbackMatch = raw.match(/FEEDBACK:\s*(.+)/s);
  const feedback = feedbackMatch ? feedbackMatch[1].trim() : "";

  return { approved, feedback };
}

// ─────────────────────────────────────────
// 3. Routing Function — The Loop Controller
// ─────────────────────────────────────────
function shouldContinue(state: typeof EssayState.State): string {
  console.log(
    `[Router] approved=${state.approved}, iterations=${state.iterations}`
  );

  // Exit condition 1: The critique approved the draft
  if (state.approved) {
    console.log("[Router] Draft approved! Exiting loop.");
    return END;
  }

  // Exit condition 2: We have tried 3 times — exit regardless
  if (state.iterations >= 3) {
    console.log("[Router] Max iterations reached. Exiting with current draft.");
    return END;
  }

  // Continue: loop back to the generate node
  console.log("[Router] Draft needs work. Looping back to generate.");
  return "generate";
}

// ─────────────────────────────────────────
// 4. Build the Graph
// ─────────────────────────────────────────
const graph = new StateGraph(EssayState)
  .addNode("generate", generate)
```

```
  .addNode("critique", critique)
  .addEdge(START, "generate")        // always start by generating
  .addEdge("generate", "critique")   // always critique after generating
  .addConditionalEdges("critique", shouldContinue);
  // After critique: either loop back to "generate" or go to END

const compiled = graph.compile();

// ─────────────────────────────────────────
// 5. Run It
// ─────────────────────────────────────────
async function main() {
  const result = await compiled.invoke({
    topic: "The impact of social media on attention spans"
  });

  console.log("\n========= FINAL ESSAY =========");
  console.log(result.draft);
  console.log(`\n(Completed in ${result.iterations} iteration(s))`);
}

main().catch(console.error);
```

Run it

```
npx tsx src/unit3/lab2.ts
```

Expected Output

```
[Node: generate] Iteration 1
[Node: generate] Draft length: 187 words
[Node: critique] Evaluating draft...
[Node: critique] Raw critique:
 VERDICT: APPROVED
 FEEDBACK:
[Router] approved=true, iterations=1
[Router] Draft approved! Exiting loop.

========= FINAL ESSAY =========
(your essay here)

(Completed in 1 iteration(s))
```

Sometimes it takes 2–3 iterations before the draft is approved. Watch the console to see the loop in action.

Key Insight — The Loop in the Graph

```
START → generate → critique → [conditional]
                                    ↓ approved=true OR iterations≥3 → END
                                    ↓ not approved → generate (LOOP BACK!)
```

The graph does not have a `while` loop in the code. The loop is built into the **graph structure** itself through the conditional edge pointing backward.

Lab 3.3: Triggering the Recursion Limit on Purpose

What you will build: A graph with a loop that never terminates, and you will observe the recursion limit error.

Why this matters: Understanding what the error looks like and how to catch it is essential for debugging production graphs.

```typescript
// src/unit3/lab3.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";

const LoopState = Annotation.Root({
  counter: Annotation<number>({
    reducer: (_, b) => b,
    default: () => 0
  })
});

// This node increments a counter — nothing else
function increment(state: typeof LoopState.State) {
  const newCount = state.counter + 1;
  console.log("[Node: increment] Counter is now:", newCount);
  return { counter: newCount };
}

// This routing function NEVER returns END — it always loops back
// This is intentionally broken to demonstrate the recursion limit
function neverStop(state: typeof LoopState.State): string {
  return "increment"; // always loop — no exit condition!
}

const graph = new StateGraph(LoopState)
  .addNode("increment", increment)
  .addEdge(START, "increment")
  .addConditionalEdges("increment", neverStop);

const compiled = graph.compile();

// ─────────────────────────────────────────────
// Test 1: Let the default recursion limit (25) kick in
// ─────────────────────────────────────────────
async function test1() {
  console.log("=== TEST 1: Default recursion limit (25) ===");
  try {
    await compiled.invoke({ counter: 0 });
  } catch (error) {
    console.error("\n>>> Recursion limit error caught:");
    console.error((error as Error).message);
  }
}

// ─────────────────────────────────────────────
// Test 2: Set a custom low recursion limit
```

```
// ──────────────────────────────────────────────
async function test2() {
  console.log("\n=== TEST 2: Custom recursion limit of 5 ===");
  try {
    await compiled.invoke(
      { counter: 0 },
      { recursionLimit: 5 }  // stop after 5 node executions
    );
  } catch (error) {
    console.error("\n>>> Recursion limit error caught:");
    console.error((error as Error).message);
  }
}

async function main() {
  await test1();
  await test2();
  console.log("\nBoth tests completed — errors were caught safely.");
}

main().catch(console.error);
```

Run it

```
npx tsx src/unit3/lab3.ts
```

Expected Output

```
=== TEST 1: Default recursion limit (25) ===
[Node: increment] Counter is now: 1
[Node: increment] Counter is now: 2
... (continues up to ~25)
>>> Recursion limit error caught:
Recursion limit of 25 reached...

=== TEST 2: Custom recursion limit of 5 ===
[Node: increment] Counter is now: 1
[Node: increment] Counter is now: 2
[Node: increment] Counter is now: 3
[Node: increment] Counter is now: 4
[Node: increment] Counter is now: 5
>>> Recursion limit error caught:
Recursion limit of 5 reached...
```

Lab 3.4: Visualizing Your Graph as a Mermaid Diagram

What you will build: Export the structure of Lab 3.2's graph as a Mermaid diagram.

Why this matters: As graphs get complex, visualizing them helps you understand the flow and catch mistakes in wiring.

```typescript
// src/unit3/lab4.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";

// Minimal version of the essay writer graph — no LLM calls needed for visualization
const EssayState = Annotation.Root({
  draft: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  approved: Annotation<boolean>({ reducer: (_, b) => b, default: () => false }),
  iterations: Annotation<number>({ reducer: (_, b) => b, default: () => 0 })
});

function generate(state: typeof EssayState.State) { return {}; }
function critique(state: typeof EssayState.State) { return {}; }
function shouldContinue(state: typeof EssayState.State): string {
  return state.approved || state.iterations >= 3 ? END : "generate";
}

const graph = new StateGraph(EssayState)
  .addNode("generate", generate)
  .addNode("critique", critique)
  .addEdge(START, "generate")
  .addEdge("generate", "critique")
  .addConditionalEdges("critique", shouldContinue);

const compiled = graph.compile();

// ─────────────────────────────────────────
// Export the Mermaid diagram
// ─────────────────────────────────────────
async function main() {
  // getGraph() returns a representation of the compiled graph
  const graphRepresentation = compiled.getGraph();

  // drawMermaid() returns a Mermaid diagram as a string
  const mermaid = graphRepresentation.drawMermaid();
  console.log("=== Mermaid Diagram ===\n");
  console.log(mermaid);
  console.log("\nPaste the above into https://mermaid.live to view it visually.");
}

main().catch(console.error);
```

Run it and view the result

```
npx tsx src/unit3/lab4.ts
```

Copy the output and paste it at **https://mermaid.live** to see a visual diagram of your graph, including the cycle.

Unit 3 Knowledge Check

Summary

Concept	What it does
`addConditionalEdges(source, fn)`	After `source` runs, calls `fn(state)` to decide the next node
Routing function	A plain function that returns the next node name as a string
Path map (3rd arg)	Translates symbolic return values to actual node names
Fan-out	One routing function can route to many possible destinations
Cycle	A conditional edge that routes back to an earlier node
Iteration counter	A state field you manually increment to track loop progress
`recursionLimit`	Hard stop set in `invoke({}, { recursionLimit: N })`

Key Takeaways

- Conditional edges make your graph dynamic. Static edges make it linear.
- Cycles are the most powerful feature of LangGraph — they enable agentic loops.
- Always include **two** exit conditions in a loop: a success condition and an iteration cap.
- `recursionLimit` is a safety net, not a loop controller. Do not rely on it as your only exit condition.
- The routing function just returns a string. The simpler it is, the easier it is to debug.

Practice Questions

1. What is the difference between a static edge and a conditional edge?
2. If your routing function returns `"nodeX"` but `nodeX` was never added to the graph, what happens?
3. You have a loop that should run at most 5 times. Where is the better place to enforce this — inside the routing function, or with `recursionLimit`? Why?
4. What does `temperature: 0` do in the ChatOpenAI constructor, and why did we use it for classification?
5. In Lab 3.2, what would happen if you removed `state.iterations >= 3` from the routing function?

Exercise

Extend Lab 3.1 to add a third classification: `"spam"`. When the message is classified as spam, route it to a new `handleSpam` node that logs ”Spam detected and discarded.” and returns `{ response: "Message discarded." }`.

Mini Project

Build a **Code Review Bot** with this loop:

1. Node `generateCode` — writes a Python function based on a user specification
2. Node `testCode` — uses the LLM to check if the code has obvious bugs
3. Routing function — if bugs found, loop back to `generateCode` with the bug list in state; if no bugs or after 4 iterations, go to `END`

UNIT 4: Command & Send API

Chapter 6: The Command Object — Routing From Inside a Node

6.1 Recap — The Problem with addConditionalEdges

In Unit 3, you learned that `addConditionalEdges` separates two concerns:

1. The node does work and updates state
2. A **separate routing function** reads state and decides where to go next

This is clean, but it has a limitation: the routing function only reads state. It cannot **also** update state. The node and the routing function are two different functions.

Sometimes you want to do both at once — update state AND decide the next node — in a single return value from inside the node itself.

Real-world analogy — The Airport Security Officer

With `addConditionalEdges`, it is like this:

1. Security officer scans your bag (the node)
2. A different person outside checks the scan results and points you to the right gate (the routing function)

With `Command`, it is like this:

1. The security officer scans your bag AND immediately tells you "Gate 12, and by the way your boarding pass needs to be updated" — one person, one decision, one action.

6.2 What is a Command?

`Command` is a special object you return from a node instead of a plain state update. It bundles two things together:

```
import { Command } from "@langchain/langgraph";

return new Command({
  update: { someField: "new value" },  // state update (same as returning a plain object)
  goto: "nextNodeName"                 // routing decision (which node to go to next)
});
```

- **`update`** — the state fields to update (same as what you normally return from a node)
- **`goto`** — the name of the next node to route to (replaces the need for an external routing function)

6.3 Minimum Example

```
function processRequest(state: typeof MyState.State) {
  const isValid = state.input.length > 0;

  // Instead of returning a plain object, return a Command
  return new Command({
    update: {
      processed: true,
      lastAction: "processRequest"
    },
    goto: isValid ? "handleValid" : "handleInvalid"
  });
}
```

This node both **updates state** and **decides the next node** in one return value. No separate routing function is needed.

6.4 What Command Does NOT Replace

Command does not replace `addConditionalEdges` in every situation. It has one constraint: when a node returns `Command`, LangGraph needs to know at compile time which nodes it might route to. You tell it this by listing them in `addConditionalEdges` with `__end__` or the path map — or by using the newer LangGraph approach where `Command` nodes are self-declaring.

In practice: if your routing logic is simple and self-contained in the node, use `Command`. If multiple nodes need to be wired together in the graph schema at build time, stick with `addConditionalEdges`.

Chapter 7: Static vs. Dynamic Routing — Choosing the Right Tool

Here is a clear decision framework:

Situation	Best Tool
The routing logic is simple and the node computes the decision	`Command` (routing from inside the node)
Multiple different nodes might need to trigger the same routing decision	`addConditionalEdges` with a shared routing
You want to see the routing logic clearly in the graph schema	`addConditionalEdges` (the routing function
The routing depends on some computation that only the node can do	`Command` (the node has the context)
You are building a subgraph or multi-agent handoff (Unit 4+)	`Command` (it is designed for this)

The Same Logic — Two Ways

Both of these accomplish the same result:

With addConditionalEdges:

```
function processNode(state) {
  // ... do work
  return { approved: true };
}

function routeFn(state) {
  return state.approved ? "approveNode" : "rejectNode";
}

graph.addConditionalEdges("processNode", routeFn);
```

With Command:

```
function processNode(state) {
  // ... do work
  const approved = true;
  return new Command({
    update: { approved },
    goto: approved ? "approveNode" : "rejectNode"
  });
}

// No addConditionalEdges needed for this node
```

The `Command` version is more compact when the routing logic belongs naturally inside the node's function.

Chapter 8: The Send API — True Parallel Fan-Out

8.1 The Problem That Send Solves

So far, all the graphs you have built process one thing at a time. State has one `draft`, one `message`, one `score`. Nodes run one after another.

But what if you have a list of things to process?

For example: you have 10 documents and you want to run sentiment analysis on all 10 at the same time — in parallel, not one by one.

With what you know so far, you would have to loop through the list and call a node 10 times sequentially. That is slow.

`Send` lets you create 10 parallel instances of the same node, each processing one document at the same time.

Analogy — A Warehouse Packing Line

Imagine you have 100 packages to seal and label. Two approaches:

1. **Sequential (no Send):** One worker seals package 1, then package 2, then package 3... takes forever.
2. **Parallel (with Send):** You send each package to a different worker station simultaneously. All 100 are processed at the same time.

`Send` is the manager who distributes packages to different worker stations.

8.2 What is Send?

`Send` is an object that says: "Execute THIS node with THIS custom state, as a parallel instance."

```
import { Send } from "@langchain/langgraph";

// Create a Send object for each item you want to process in parallel
const sends = myItems.map(item =>
  new Send(
    "processItem",    // the node to invoke
    { item: item }    // the state to pass to that node instance
  )
);

return sends; // return an array of Send objects from a router node
```

When you return an array of `Send` objects from a node, LangGraph creates multiple parallel executions of the target node — one for each `Send` object — each with its own piece of state.

8.3 How LangGraph Executes Send

Here is the key insight: **each `Send` spawns an independent sub-execution of the target node.** They run in parallel and their results are merged back into the main state via a reducer.

This is why your state reducer matters so much when using `Send`. If you are collecting results from 10 parallel node executions, your reducer needs to aggregate them (e.g., append to an array).

```
const BatchState = Annotation.Root({
  documents: Annotation<string[]>({
    reducer: (_, b) => b,   // input list — replaced once
    default: () => []
  }),
  results: Annotation<string[]>({
    reducer: (existing, incoming) => [...existing, ...incoming],  // APPEND
    // Why append: each parallel Send returns one result,
    // and we want to collect ALL of them into the results array
    default: () => []
  })
});
```

8.4 The Router Node Pattern with Send

The typical pattern is:

`START → routerNode → [Send × N] → processNode (× N, in parallel) → aggregateNode → END`

1. `routerNode` — reads the list from state, creates one `Send` per item
2. `processNode` — runs N times in parallel, each with one item
3. `aggregateNode` — combines all the parallel results (optional, if you need post-processing)

```
function routerNode(state: typeof BatchState.State): Send[] {
  // Create one parallel execution per document
  return state.documents.map(doc =>
    new Send("processDocument", { singleDoc: doc })
  );
}
```

Chapter 9: Map-Reduce — The Most Powerful Pattern in LangGraph

9.1 What is Map-Reduce?

Map-Reduce is a famous pattern from distributed computing (Google used it to index the entire web). In LangGraph, it refers to:

1. **Map phase:** Take a list of items and process each one independently (in parallel using `Send`)
2. **Reduce phase:** Gather all the individual results and combine them into a single final output

Analogy — Crowdsourcing a Report

Imagine you need to research 20 different countries for a global market report:

- **Map:** You assign one researcher per country. They all work in parallel.
- **Reduce:** When all researchers are done, you combine their reports into one final document.

This is exactly what map-reduce does in LangGraph.

9.2 The Full Map-Reduce Graph

```
START
  → mapRouter           (creates one Send per chunk)
  → summarizeChunk × N  (runs in parallel for each chunk)
  → combineResults      (aggregates all summaries)
  → END
```

9.3 State Design for Map-Reduce

The state design is critical. You need:

- A field for the INPUT list (the chunks to process)
- A field for the INTERMEDIATE results (one per parallel execution) — needs an **append** reducer
- A field for the FINAL output

```
const MapReduceState = Annotation.Root({
  // Input: the list of chunks to process
  chunks: Annotation<string[]>({
    reducer: (_, b) => b,
    default: () => []
  }),
  // Intermediate: one summary per chunk (collected via append reducer)
  summaries: Annotation<string[]>({
    reducer: (existing, incoming) => [...existing, ...incoming],
    default: () => []
  }),
  // Final output: the combined summary of all summaries
  finalSummary: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  })
});
```

9.4 Why the Append Reducer is Essential

When 5 parallel executions each return `{ summaries: ["my summary"] }`, LangGraph calls your reducer 5 times:

```
reducer([], ["summary1"])     → ["summary1"]
reducer(["summary1"], ["summary2"]) → ["summary1", "summary2"]
...
```

Without the append reducer (`(existing, incoming) => [...existing, ...incoming]`), each parallel result would overwrite the previous one and you would only see the last result.

Practical Labs — Unit 4

Lab 4.1: Refactor a Conditional-Edge Graph to Use Command

What you will build: Take the content moderation router from Lab 3.1 and rewrite it so the `classifyMessage` node returns a `Command` instead of a plain object, eliminating the need for a

separate routing function.

What you will learn:

- The difference between the two approaches side by side
- When `Command` is more readable
- How `Command` simplifies code without changing behavior

```typescript
// src/unit4/lab1.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { Command } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const ModerationState = Annotation.Root({
  userMessage: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  classification: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  response: Annotation<string>({ reducer: (_, b) => b, default: () => "" })
});

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

// ─────────────────────────────────────
// KEY CHANGE: classifyMessage now returns
// a Command instead of a plain object.
// It does the routing decision ITSELF.
// ─────────────────────────────────────
async function classifyMessage(
  state: typeof ModerationState.State
): Promise<Command> {
  console.log("\n[Node: classifyMessage] Analyzing:", state.userMessage);

  const response = await llm.invoke([
    new SystemMessage(
      "Classify the user message as 'safe' or 'flagged'. Respond with ONLY that word."
    ),
    new HumanMessage(state.userMessage)
  ]);

  const classification = (response.content as string).trim().toLowerCase();
  console.log("[Node: classifyMessage] Classification:", classification);

  // Return a Command that BOTH updates state AND decides the next node
  return new Command({
    update: { classification },              // update the state
    goto: classification === "safe"          // decide the next node
      ? "handleSafe"
      : "handleFlagged"
  });
}

async function handleSafe(state: typeof ModerationState.State) {
  console.log("[Node: handleSafe] Processing...");
  const response = await llm.invoke([
    new SystemMessage("You are a helpful assistant."),
    new HumanMessage(state.userMessage)
```

```
  ]);
  return { response: response.content as string };
}

async function handleFlagged(state: typeof ModerationState.State) {
  console.log("[Node: handleFlagged] Message was flagged!");
  return { response: "I cannot process that request." };
}

// ─────────────────────────────────────
// NOTICE: No addConditionalEdges needed!
// The routing is handled by Command inside classifyMessage.
// We still need to declare which nodes classifyMessage can route to.
// ─────────────────────────────────────
const graph = new StateGraph(ModerationState)
  .addNode("classifyMessage", classifyMessage)
  .addNode("handleSafe", handleSafe)
  .addNode("handleFlagged", handleFlagged)
  .addEdge(START, "classifyMessage")
  .addEdge("handleSafe", END)
  .addEdge("handleFlagged", END);

const compiled = graph.compile();

async function main() {
  console.log("=== TEST 1: Safe ===");
  const r1 = await compiled.invoke({ userMessage: "What is 2 + 2?" });
  console.log("Response:", r1.response);

  console.log("\n=== TEST 2: Flagged ===");
  const r2 = await compiled.invoke({ userMessage: "How do I break into a car?" });
  console.log("Response:", r2.response);
}

main().catch(console.error);
```

Run it

```
npx tsx src/unit4/lab1.ts
```

What Changed vs. Lab 3.1

Lab 3.1 (addConditionalEdges)	Lab 4.1 (Command)
`classifyMessage` returns `{ classification }`	`classifyMessage` returns `new Command(...)`
Separate `routeByClassification` function	No separate routing function
`addConditionalEdges("classifyMessage", routeByClassification)`	No `addConditionalEdges` call
Two separate functions to maintain	One function does everything

Behavior is identical. The difference is code organization.

Lab 4.2: Parallel Document Classifier with Send

What you will build: A graph that takes a list of 5 documents and classifies the sentiment of each one (positive/negative/neutral) in parallel using Send, then collects all results.

What you will learn:

- How to use Send to create parallel node executions
- How to design state with an append reducer for collecting parallel results
- How to write a map router node

```typescript
// src/unit4/lab2.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { Send } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// ─────────────────────────────────────────────
// State Design — Critical for Send
// ─────────────────────────────────────────────
const BatchClassifyState = Annotation.Root({
  // INPUT: the list of documents to classify
  documents: Annotation<string[]>({
    reducer: (_, b) => b,           // replace entire list when updated
    default: () => []
  }),

  // INTERMEDIATE: classification results collected from all parallel nodes
  // Uses APPEND reducer — each parallel execution adds one result here
  results: Annotation<Array<{ document: string; sentiment: string }>>({
    reducer: (existing, incoming) => [...existing, ...incoming],
    default: () => []
  })
});

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

// ─────────────────────────────────────────────
// NODE 1: Map Router
// This node reads the documents list and creates
// one Send() per document — launching parallel executions
// ─────────────────────────────────────────────
function mapRouter(state: typeof BatchClassifyState.State): Send[] {
  console.log(
    `[Node: mapRouter] Launching ${state.documents.length} parallel classifiers`
  );

  // Create one Send per document
  // Each Send says: "run the 'classifySingle' node with THIS document's data"
  return state.documents.map((doc, index) =>
    new Send("classifySingle", {
      // We pass a subset of state to each parallel execution
      // The parallel node only sees what we give it here
      documents: [doc],   // pass as single-item array so the node knows its schema
      results: []         // initialize empty — each instance starts fresh
    })
```

```
  );
}

// ─────────────────────────────────────────────
// NODE 2: classifySingle
// This runs N times in parallel, once per document.
// Each instance receives a different document via Send.
// ─────────────────────────────────────────────
async function classifySingle(
  state: typeof BatchClassifyState.State
) {
  const doc = state.documents[0]; // we passed a single-item array per Send
  console.log(
    `[Node: classifySingle] Classifying: "${doc.substring(0, 50)}..."`
  );

  const response = await llm.invoke([
    new SystemMessage(
      "Classify the sentiment of this text. " +
      "Respond with ONLY one word: 'positive', 'negative', or 'neutral'."
    ),
    new HumanMessage(doc)
  ]);

  const sentiment = (response.content as string).trim().toLowerCase();
  console.log(`[Node: classifySingle] Sentiment: ${sentiment}`);

  // Return a result object — the append reducer will collect this
  // from all N parallel executions
  return {
    results: [{ document: doc, sentiment }]
  };
}

// ─────────────────────────────────────────────
// NODE 3: displayResults (Reduce/Aggregate phase)
// After all parallel executions finish,
// this node receives the fully merged state.results array
// ─────────────────────────────────────────────
function displayResults(state: typeof BatchClassifyState.State) {
  console.log("\n[Node: displayResults] All parallel jobs complete!");
  console.log(`Total results collected: ${state.results.length}`);
  return {};
}

// ─────────────────────────────────────────────
// Build the Graph
// ─────────────────────────────────────────────
const graph = new StateGraph(BatchClassifyState)
  .addNode("mapRouter", mapRouter)
  .addNode("classifySingle", classifySingle)
  .addNode("displayResults", displayResults)
  .addEdge(START, "mapRouter")
  // After all parallel classifySingle executions complete, go to displayResults
  .addEdge("classifySingle", "displayResults")
  .addEdge("displayResults", END);
```

```
const compiled = graph.compile();

// ──────────────────────────────────────────
// Run it
// ──────────────────────────────────────────
async function main() {
  const testDocuments = [
    "I absolutely loved this product! It exceeded all my expectations.",
    "The service was terrible. I waited 2 hours and nobody helped me.",
    "The package arrived on Tuesday as scheduled.",
    "This is the worst purchase I have ever made. Total waste of money.",
    "The coffee this morning was fine. Nothing special."
  ];

  console.log("Starting parallel classification of", testDocuments.length,
   ↪  "documents...\n");

  const result = await compiled.invoke(
    { documents: testDocuments },
    { recursionLimit: 50 }  // increase limit because parallel executions count
 ↪  individually
  );

  console.log("\n========= CLASSIFICATION RESULTS =========");
  result.results.forEach((r: { document: string; sentiment: string }, i: number) => {
    console.log(`\n[${i + 1}] Sentiment: ${r.sentiment.toUpperCase()}`);
    console.log(`    Text: "${r.document.substring(0, 60)}..."`);
  });
}

main().catch(console.error);
```

Run it

```
npx tsx src/unit4/lab2.ts
```

Expected Output

```
Starting parallel classification of 5 documents...

[Node: mapRouter] Launching 5 parallel classifiers
[Node: classifySingle] Classifying: "I absolutely loved this product!..."
[Node: classifySingle] Classifying: "The service was terrible..."
[Node: classifySingle] Classifying: "The package arrived on Tuesday..."
[Node: classifySingle] Classifying: "This is the worst purchase..."
[Node: classifySingle] Classifying: "The coffee this morning was fine..."

(all 5 run roughly in parallel)

[Node: displayResults] All parallel jobs complete!
Total results collected: 5

========= CLASSIFICATION RESULTS =========
[1] Sentiment: POSITIVE
```

```
[2] Sentiment: NEGATIVE
...
```

> **Note:** The order of `classifySingle` logs may vary — they are running in parallel so whichever finishes first logs first.

Why recursionLimit: 50?

With 5 parallel `classifySingle` executions, LangGraph counts each one toward the total step count. The default limit of 25 could be hit. As a rule: `recursionLimit` = number of parallel branches × average steps per branch + buffer.

Lab 4.3: Map-Reduce Summarization Pipeline

What you will build: A full map-reduce pipeline that splits a long text into chunks, summarizes each chunk in parallel (map phase), then combines all summaries into one final summary (reduce phase).

What you will learn:

- Full map-reduce pattern end-to-end
- State design with multiple phases
- How the reduce node receives the aggregated results

```typescript
// src/unit4/lab3.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { Send } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// ─────────────────────────────────────────────
// State
// ─────────────────────────────────────────────
const SummarizationState = Annotation.Root({
  // The full text to summarize
  fullText: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
  }),
  // The chunks we split the text into
  chunks: Annotation<string[]>({
    reducer: (_, b) => b,
    default: () => []
  }),
  // Individual chunk summaries — APPEND reducer for parallel collection
  chunkSummaries: Annotation<string[]>({
    reducer: (existing, incoming) => [...existing, ...incoming],
    default: () => []
  }),
  // The final combined summary
  finalSummary: Annotation<string>({
    reducer: (_, b) => b,
    default: () => ""
```

```
  })
});

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

// Helper: Split text into roughly equal chunks by word count
function splitIntoChunks(text: string, wordsPerChunk: number): string[] {
  const words = text.split(" ");
  const chunks: string[] = [];
  for (let i = 0; i < words.length; i += wordsPerChunk) {
    chunks.push(words.slice(i, i + wordsPerChunk).join(" "));
  }
  return chunks;
}

// ─────────────────────────────────────────────
// PHASE 0: Splitter — Prepare chunks
// ─────────────────────────────────────────────
function splitText(state: typeof SummarizationState.State) {
  const chunks = splitIntoChunks(state.fullText, 100); // 100 words per chunk
  console.log(`[Node: splitText] Split into ${chunks.length} chunks`);
  return { chunks };
}

// ─────────────────────────────────────────────
// PHASE 1: Map Router — Launch parallel summarization
// ─────────────────────────────────────────────
function mapRouter(state: typeof SummarizationState.State): Send[] {
  console.log(`[Node: mapRouter] Launching ${state.chunks.length} parallel summarizers`);
  return state.chunks.map((chunk, index) =>
    new Send("summarizeChunk", {
      fullText: "",
      chunks: [chunk],       // one chunk per parallel execution
      chunkSummaries: [],
      finalSummary: ""
    })
  );
}

// ─────────────────────────────────────────────
// PHASE 1: Summarize Chunk — runs N times in parallel
// ─────────────────────────────────────────────
async function summarizeChunk(state: typeof SummarizationState.State) {
  const chunk = state.chunks[0];
  console.log(`[Node: summarizeChunk] Summarizing chunk: "${chunk.substring(0,
  ↪  40)}..."`);

  const response = await llm.invoke([
    new SystemMessage("Summarize the following text in 1-2 sentences."),
    new HumanMessage(chunk)
  ]);

  const summary = response.content as string;
  console.log(`[Node: summarizeChunk] Summary: "${summary.substring(0, 60)}..."`);

  return { chunkSummaries: [summary] };
}
```

```
// ─────────────────────────────────────────────
// PHASE 2: Combine Summaries — the REDUCE step
// After all parallel summarizeChunk runs complete,
// this node receives state.chunkSummaries with ALL summaries collected.
// ─────────────────────────────────────────────
async function combineSummaries(state: typeof SummarizationState.State) {
  console.log(
    `\n[Node: combineSummaries] Combining ${state.chunkSummaries.length} chunk summaries`
  );

  const combinedInput = state.chunkSummaries
    .map((s, i) => `Section ${i + 1}: ${s}`)
    .join("\n\n");

  const response = await llm.invoke([
    new SystemMessage(
      "You are given summaries of different sections of a document. " +
      "Combine them into one coherent overall summary in 3-4 sentences."
    ),
    new HumanMessage(combinedInput)
  ]);

  return { finalSummary: response.content as string };
}

// ─────────────────────────────────────────────
// Build the Graph
// ─────────────────────────────────────────────
const graph = new StateGraph(SummarizationState)
  .addNode("splitText", splitText)
  .addNode("mapRouter", mapRouter)
  .addNode("summarizeChunk", summarizeChunk)
  .addNode("combineSummaries", combineSummaries)
  .addEdge(START, "splitText")
  .addEdge("splitText", "mapRouter")
  .addEdge("summarizeChunk", "combineSummaries")  // after ALL parallel chunks finish
  .addEdge("combineSummaries", END);

const compiled = graph.compile();

// ─────────────────────────────────────────────
// Run it
// ─────────────────────────────────────────────
async function main() {
  const longArticle = `
    Artificial intelligence has undergone remarkable transformation over the past decade.
    What began as theoretical computer science has become the defining technology of our
 ↪  era.
    Large language models trained on vast corpora of text have demonstrated capabilities
    that were once thought to be uniquely human: writing poetry, explaining complex
 ↪  concepts,
    translating languages, and generating functional computer code.

    The implications for the workforce are profound. Economists predict that AI will
 ↪  automate
```

```
    repetitive cognitive tasks in the same way that industrial machinery automated
↪   physical
    labour in the 20th century. While some roles will become obsolete, new categories of
    work are emerging that require human-AI collaboration, critical thinking, and creative
    problem solving. The transition will not be painless, but history suggests that
↪   technological
    revolutions ultimately create more economic value than they destroy.

    In healthcare, AI diagnostic tools are already matching or exceeding specialist
↪   physicians
    at detecting certain cancers from imaging data. Drug discovery timelines that once
↪   spanned
    a decade can now be compressed to months. Personalized treatment plans informed by
↪   genetic
    data and historical outcomes represent a fundamental shift in how medicine is
↪   practiced.

    The ethical dimensions of this shift demand careful attention. Questions of
↪   algorithmic bias,
    data privacy, accountability for automated decisions, and the concentration of AI
↪   power
    among a small number of corporations are not merely technical problems — they are
↪   political
    and moral challenges that require democratic deliberation. How societies answer these
↪   questions
    will shape the character of the AI era for generations.
  `.trim();

  console.log("Starting map-reduce summarization...\n");

  const result = await compiled.invoke(
    { fullText: longArticle },
    { recursionLimit: 100 }
  );

  console.log("\n========= FINAL SUMMARY =========");
  console.log(result.finalSummary);
  console.log(`\nProcessed ${result.chunkSummaries.length} chunks in parallel.`);
}

main().catch(console.error);
```

Run it

```
npx tsx src/unit4/lab3.ts
```

The Data Flow Visualized

```
fullText (1 big string)
    ↓
[splitText]
    ↓
chunks: ["chunk1", "chunk2", "chunk3", "chunk4"]
    ↓
[mapRouter] → Send("summarizeChunk", chunk1)
```

```
            → Send("summarizeChunk", chunk2)  ← all run in parallel
            → Send("summarizeChunk", chunk3)
            → Send("summarizeChunk", chunk4)
                ↓ (each returns chunkSummaries: ["one summary"])
                ↓ (append reducer collects all 4)
chunkSummaries: ["summary1", "summary2", "summary3", "summary4"]
    ↓
[combineSummaries]
    ↓
finalSummary: "The full combined summary..."
```

Lab 4.4: Command with Both update and goto

What you will build: A pipeline where each node makes a routing decision AND performs a state update simultaneously using Command — demonstrating the full power of the object.

```typescript
// src/unit4/lab4.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { Command } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// Simulates a multi-stage job application pipeline
const ApplicationState = Annotation.Root({
  candidateName: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  yearsExperience: Annotation<number>({ reducer: (_, b) => b, default: () => 0 }),
  technicalScore: Annotation<number>({ reducer: (_, b) => b, default: () => 0 }),
  culturalScore: Annotation<number>({ reducer: (_, b) => b, default: () => 0 }),
  decision: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  decisionReason: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  stagesCompleted: Annotation<string[]>({
    reducer: (existing, incoming) => [...existing, ...incoming],
    default: () => []
  })
});

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });

// ─────────────────────────────────────────
// Stage 1: Resume Screen
// Updates state AND routes to interview OR reject
// ─────────────────────────────────────────
async function resumeScreen(
  state: typeof ApplicationState.State
): Promise<Command> {
  console.log(`\n[Stage 1: Resume Screen] Evaluating ${state.candidateName}`);

  const response = await llm.invoke([
    new SystemMessage(
      "You are a recruiter. Evaluate this candidate and respond ONLY with a JSON object:
      ↪  " +
      '{"technicalScore": (0-100), "pass": true/false}'
```

```
    ),
    new HumanMessage(
      `Candidate: ${state.candidateName}, Years of Experience: ${state.yearsExperience}`
    )
  ]);

  let parsed: { technicalScore: number; pass: boolean };
  try {
    const clean = (response.content as string).replace(/```json|```/g, "").trim();
    parsed = JSON.parse(clean);
  } catch {
    parsed = { technicalScore: 50, pass: state.yearsExperience >= 3 };
  }

  console.log(`[Stage 1] Technical score: ${parsed.technicalScore}, Pass:
  ↪  ${parsed.pass}`);

  // Command: update state AND decide next stage
  return new Command({
    update: {
      technicalScore: parsed.technicalScore,
      stagesCompleted: ["resume_screen"]  // append reducer adds this
    },
    goto: parsed.pass ? "culturalInterview" : "reject"
  });
}

// ─────────────────────────────────────────
// Stage 2: Cultural Interview
// Updates state AND routes to offer OR reject
// ─────────────────────────────────────────
async function culturalInterview(
  state: typeof ApplicationState.State
): Promise<Command> {
  console.log(`[Stage 2: Cultural Interview] Evaluating ${state.candidateName}`);

  const response = await llm.invoke([
    new SystemMessage(
      "You are a culture interviewer. Respond ONLY with JSON: " +
      '{"culturalScore": (0-100), "pass": true/false}'
    ),
    new HumanMessage(
      `Candidate: ${state.candidateName} with technical score of ${state.technicalScore}`
    )
  ]);

  let parsed: { culturalScore: number; pass: boolean };
  try {
    const clean = (response.content as string).replace(/```json|```/g, "").trim();
    parsed = JSON.parse(clean);
  } catch {
    parsed = { culturalScore: 75, pass: true };
  }

  console.log(`[Stage 2] Cultural score: ${parsed.culturalScore}, Pass: ${parsed.pass}`);

  return new Command({
```

```
    update: {
      culturalScore: parsed.culturalScore,
      stagesCompleted: ["cultural_interview"]
    },
    goto: parsed.pass ? "makeOffer" : "reject"
  });
}

// Stage 3a: Make an offer
function makeOffer(state: typeof ApplicationState.State) {
  console.log(`[Stage 3: Make Offer] Offering position to ${state.candidateName}!`);
  return {
    decision: "HIRED",
    decisionReason: `Tech: ${state.technicalScore}, Cultural: ${state.culturalScore}`,
    stagesCompleted: ["offer_made"]
  };
}

// Stage 3b: Reject
function reject(state: typeof ApplicationState.State) {
  console.log(`[Stage 3: Reject] Rejecting ${state.candidateName}`);
  return {
    decision: "REJECTED",
    decisionReason: "Did not meet the minimum threshold.",
    stagesCompleted: ["rejected"]
  };
}

const graph = new StateGraph(ApplicationState)
  .addNode("resumeScreen", resumeScreen)
  .addNode("culturalInterview", culturalInterview)
  .addNode("makeOffer", makeOffer)
  .addNode("reject", reject)
  .addEdge(START, "resumeScreen")
  .addEdge("makeOffer", END)
  .addEdge("reject", END);

const compiled = graph.compile();

async function main() {
  const candidates = [
    { candidateName: "Alice Chen", yearsExperience: 8 },
    { candidateName: "Bob Smith", yearsExperience: 1 }
  ];

  for (const candidate of candidates) {
    console.log(`\n${"=".repeat(50)}`);
    console.log(`Processing: ${candidate.candidateName}`);
    const result = await compiled.invoke(candidate);
    console.log(`\nFINAL DECISION: ${result.decision}`);
    console.log(`Reason: ${result.decisionReason}`);
    console.log(`Stages completed: ${result.stagesCompleted.join(" → ")}`);
  }
}

main().catch(console.error);
```

Run it

```
npx tsx src/unit4/lab4.ts
```

This demonstrates `Command` at its most useful: each stage node simultaneously updates state with its evaluation scores AND routes to the appropriate next stage — all in one clean return value.

Unit 4 Knowledge Check

Summary

Concept	What it does
`Command({ update, goto })`	Returns from a node to update state AND route to a specific next node at once
`Send("nodeName", state)`	Creates one parallel execution of `nodeName` with the given state
Map Router	A node that returns `state.list.map(item => new Send(...))`
Append reducer	`(existing, incoming) => [...existing, ...incoming]` — collects results from parallel
Map-Reduce	Map = `Send` for parallel work; Reduce = aggregation node after parallel phase

Key Takeaways

- `Command` is ideal when the routing decision is naturally made inside the node itself (it has the context needed to decide).
- `Send` is for parallelism — processing a list of items simultaneously rather than sequentially.
- The append reducer is the key that makes `Send` work correctly. Without it, parallel results overwrite each other.
- Always increase `recursionLimit` when using `Send` — parallel executions each count against the total.
- `Command` and `Send` can be combined: a map router can return `Command` objects that use `Send` internally.

Practice Questions

1. What is the main difference between `Command` and returning a plain object from a node?
2. When you return `new Send("myNode", stateSlice)` from a router, what does LangGraph do with it?
3. Why does the append reducer `(existing, incoming) => [...existing, ...incoming]` work correctly when 5 parallel nodes each return one result?
4. If you are processing a list of 20 documents in parallel and each `summarizeChunk` runs 2 nodes internally, what `recursionLimit` would you need approximately?
5. Describe a real-world scenario where you would use `Command` instead of `addConditionalEdges`.

Exercise

Modify Lab 4.2 to add a fourth field to each result: `"confidence"` (a number 0-100). Have the `classifySingle` node also return a confidence score from the LLM and include it in the results array.

Mini Project

Build a **Parallel Research Pipeline** that:

1. Takes a list of 3 research questions
2. Uses `Send` to run 3 parallel research nodes (each uses the LLM to write a 2-sentence answer to one question)
3. Collects all answers using an append reducer
4. Runs a final "compile" node that formats all 3 answers into a structured report

---

Phase 2 Final Project: Intelligent Article Review Pipeline

This project combines everything you learned in Phase 2 into one complete system.

What You Will Build

A content pipeline that:

1. Takes a list of article drafts
2. Routes each article to a sentiment classifier (Phase 1 skill: conditional edges)
3. Negative/Controversial articles go through a 3-iteration self-correction loop (Phase 1 skill: cycles)
4. All articles are processed in parallel using Send (Phase 2 skill: Send)
5. Results are aggregated in a map-reduce reduce node (Phase 2 skill: map-reduce)
6. A final report is generated using Command routing (Phase 2 skill: Command)

Architecture

```
START
  → batchRouter (Send × N articles)
    → [For each article, in parallel:]
      → classifyTone (conditional edge → positive path OR sensitiveLoop)
        → [Positive path] → formatForPublish
        → [Sensitive path] → sensitiveLoop:
            → reviseArticle → checkRevision → [loop or exit]
    → [All articles complete]
  → generateReport (Command routing)
    → [if all good] → publishAll → END
    → [if issues] → flagForManual → END
```

Full Implementation

```ts
// src/final-project/pipeline.ts
import "dotenv/config";
import { StateGraph, START, END } from "@langchain/langgraph";
import { Annotation } from "@langchain/langgraph";
import { Send, Command } from "@langchain/langgraph";
import { ChatOpenAI } from "@langchain/openai";
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

// ─────────────────────────────────────────
```

```typescript
// Types and State
// ─────────────────────────────────────────────
interface ArticleResult {
  title: string;
  finalContent: string;
  tone: string;
  revisionsNeeded: number;
  status: "published" | "flagged" | "rejected";
}

const PipelineState = Annotation.Root({
  // Input: list of article drafts
  articleDrafts: Annotation<Array<{ title: string; content: string }>>({
    reducer: (_, b) => b,
    default: () => []
  }),
  // For single-article processing (used in parallel nodes)
  currentTitle: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  currentContent: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  tone: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  revisionCount: Annotation<number>({ reducer: (_, b) => b, default: () => 0 }),
  revisionFeedback: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  revisionApproved: Annotation<boolean>({ reducer: (_, b) => b, default: () => false }),
  // Output: collected results from all parallel branches (append reducer)
  processedArticles: Annotation<ArticleResult[]>({
    reducer: (existing, incoming) => [...existing, ...incoming],
    default: () => []
  }),
  // Final report
  reportSummary: Annotation<string>({ reducer: (_, b) => b, default: () => "" }),
  pipelineDecision: Annotation<string>({ reducer: (_, b) => b, default: () => "" })
});

const llm = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0 });
const writer = new ChatOpenAI({ model: "gpt-4o-mini", temperature: 0.7 });

// ─────────────────────────────────────────────
// STAGE 1: Batch Router
// Fans out all articles to parallel processing
// ─────────────────────────────────────────────
function batchRouter(state: typeof PipelineState.State): Send[] {
  console.log(`\n[BatchRouter] Dispatching ${state.articleDrafts.length} articles for
  ↪  parallel processing`);

  return state.articleDrafts.map(article =>
    new Send("classifyTone", {
      articleDrafts: [],
      currentTitle: article.title,
      currentContent: article.content,
      tone: "",
      revisionCount: 0,
      revisionFeedback: "",
      revisionApproved: false,
      processedArticles: [],
      reportSummary: "",
      pipelineDecision: ""
    })
```

```
  );
}

// ─────────────────────────────────────────────
// STAGE 2: Classify Tone (runs N times in parallel)
// ─────────────────────────────────────────────
async function classifyTone(
  state: typeof PipelineState.State
): Promise<Command> {
  console.log(`\n[ClassifyTone] Processing: "${state.currentTitle}"`);

  const response = await llm.invoke([
    new SystemMessage(
      "Classify the tone of this article. " +
      "Respond ONLY with: 'positive', 'neutral', or 'sensitive'"
    ),
    new HumanMessage(`Title: ${state.currentTitle}\n\nContent: ${state.currentContent}`)
  ]);

  const tone = (response.content as string).trim().toLowerCase();
  console.log(`[ClassifyTone] Tone for "${state.currentTitle}": ${tone}`);

  return new Command({
    update: { tone },
    goto: (tone === "positive" || tone === "neutral") ? "formatForPublish" :
     ↪  "reviseArticle"
  });
}

// ─────────────────────────────────────────────
// STAGE 3a: Format for Publish (positive/neutral articles)
// ─────────────────────────────────────────────
function formatForPublish(state: typeof PipelineState.State) {
  console.log(`[FormatForPublish] Article approved: "${state.currentTitle}"`);
  return {
    processedArticles: [{
      title: state.currentTitle,
      finalContent: state.currentContent,
      tone: state.tone,
      revisionsNeeded: 0,
      status: "published" as const
    }]
  };
}

// ─────────────────────────────────────────────
// STAGE 3b: Revise Article (sensitive articles — enters a loop)
// ─────────────────────────────────────────────
async function reviseArticle(state: typeof PipelineState.State) {
  console.log(
    `[ReviseArticle] Revising "${state.currentTitle}" (attempt ${state.revisionCount +
     ↪  1})`
  );

  const prompt = state.revisionCount === 0
    ? `Rewrite this article to be more balanced and constructive. Keep the core
     ↪  message.\n\nTitle: ${state.currentTitle}\n\nContent: ${state.currentContent}`
```

```
    : `Revise again based on feedback: ${state.revisionFeedback}\n\nCurrent content:
     ↪  ${state.currentContent}`;

  const response = await writer.invoke([
    new SystemMessage("You are an editor. Rewrite articles to be professional and
     ↪  balanced."),
    new HumanMessage(prompt)
  ]);

  return {
    currentContent: response.content as string,
    revisionCount: state.revisionCount + 1
  };
}

// ─────────────────────────────────────────────
// STAGE 3c: Check Revision (loop controller)
// ─────────────────────────────────────────────
async function checkRevision(state: typeof PipelineState.State) {
  console.log(`[CheckRevision] Checking revision ${state.revisionCount} of
   ↪  "${state.currentTitle}"`);

  const response = await llm.invoke([
    new SystemMessage(
      "Is this article now balanced and professional? " +
      "Respond ONLY with JSON: {\"approved\": true/false, \"feedback\": \"reason if not
       ↪  approved\"}"
    ),
    new HumanMessage(state.currentContent)
  ]);

  let parsed: { approved: boolean; feedback: string };
  try {
    const clean = (response.content as string).replace(/```json|```/g, "").trim();
    parsed = JSON.parse(clean);
  } catch {
    parsed = { approved: state.revisionCount >= 2, feedback: "Max revisions" };
  }

  console.log(`[CheckRevision] Approved: ${parsed.approved}`);
  return { revisionApproved: parsed.approved, revisionFeedback: parsed.feedback };
}

// Routing function for the revision loop
function revisionRouter(state: typeof PipelineState.State): string {
  if (state.revisionApproved) return "saveRevised";
  if (state.revisionCount >= 3) return "saveRevised"; // exit after 3 attempts
  return "reviseArticle"; // loop back
}

// ─────────────────────────────────────────────
// STAGE 3d: Save Revised Article
// ─────────────────────────────────────────────
function saveRevised(state: typeof PipelineState.State) {
  const status: "published" | "flagged" = state.revisionApproved ? "published" :
   ↪  "flagged";
  console.log(
```

```
    `[SaveRevised] "${state.currentTitle}" saved with status: ${status} ` +
    `after ${state.revisionCount} revision(s)`
  );
  return {
    processedArticles: [{
      title: state.currentTitle,
      finalContent: state.currentContent,
      tone: state.tone,
      revisionsNeeded: state.revisionCount,
      status
    }]
  };
}

// ─────────────────────────────────────────────
// STAGE 4: Generate Report + Route with Command
// ─────────────────────────────────────────────
async function generateReport(
  state: typeof PipelineState.State
): Promise<Command> {
  console.log(`\n[GenerateReport] Compiling results for
   ↪  ${state.processedArticles.length} articles`);

  const publishedCount = state.processedArticles.filter(a => a.status ===
   ↪  "published").length;
  const flaggedCount = state.processedArticles.filter(a => a.status ===
   ↪  "flagged").length;

  const summary = [
    `PIPELINE REPORT`,
    `Total articles processed: ${state.processedArticles.length}`,
    `Published: ${publishedCount}`,
    `Flagged for manual review: ${flaggedCount}`,
    ``,
    `Details:`,
    ...state.processedArticles.map(a =>
      `  - "${a.title}" [${a.status.toUpperCase()}] | Tone: ${a.tone} | Revisions:
       ↪  ${a.revisionsNeeded}`
    )
  ].join("\n");

  // Route to publishAll if all articles passed, otherwise flagForManual
  const allPassed = flaggedCount === 0;

  return new Command({
    update: {
      reportSummary: summary,
      pipelineDecision: allPassed ? "PUBLISH ALL" : "MANUAL REVIEW NEEDED"
    },
    goto: allPassed ? "publishAll" : "flagForManual"
  });
}

function publishAll(state: typeof PipelineState.State) {
  console.log("[PublishAll] All articles cleared for publication!");
  return {};
}
```

```
function flagForManual(state: typeof PipelineState.State) {
  console.log("[FlagForManual] Some articles need manual review.");
  return {};
}

// ─────────────────────────────────────────
// Build the Final Graph
// ─────────────────────────────────────────
const graph = new StateGraph(PipelineState)
  // Nodes
  .addNode("batchRouter", batchRouter)
  .addNode("classifyTone", classifyTone)
  .addNode("formatForPublish", formatForPublish)
  .addNode("reviseArticle", reviseArticle)
  .addNode("checkRevision", checkRevision)
  .addNode("saveRevised", saveRevised)
  .addNode("generateReport", generateReport)
  .addNode("publishAll", publishAll)
  .addNode("flagForManual", flagForManual)

  // Edges
  .addEdge(START, "batchRouter")

  // Parallel branches: after all classifyTone complete, paths converge at the aggregators
  .addEdge("formatForPublish", "generateReport")
  .addEdge("saveRevised", "generateReport")

  // Revision loop edges
  .addEdge("reviseArticle", "checkRevision")
  .addConditionalEdges("checkRevision", revisionRouter)

  // Final stage
  .addEdge("publishAll", END)
  .addEdge("flagForManual", END);

const compiled = graph.compile();

// ─────────────────────────────────────────
// Run the Pipeline
// ─────────────────────────────────────────
async function main() {
  const articleDrafts = [
    {
      title: "New Park Opens in Downtown",
      content: "The city unveiled a beautiful new green space today. Residents celebrated
↪  with picnics and live music. The 5-acre park includes playgrounds, walking trails,
↪  and a community garden."
    },
    {
      title: "Tech Layoffs: Thousands Unemployed",
      content: "Corporations are ruthlessly destroying lives. Greedy executives are
↪  cutting thousands of jobs to pad their own pockets while ordinary workers suffer.
↪  This is corporate cruelty at its worst."
    },
    {
      title: "New Public Library Branch Opens",
```

```
      content: "The city has opened a new library branch in the east district, providing
↪  access to books, digital resources, and community programs for residents in the
↪  area."
    }
  ];

  console.log("Starting Intelligent Article Review Pipeline...");
  console.log(`Processing ${articleDrafts.length} articles\n`);

  const result = await compiled.invoke(
    { articleDrafts },
    { recursionLimit: 150 }
  );

  console.log("\n" + "=".repeat(60));
  console.log(result.reportSummary);
  console.log("\nPIPELINE DECISION:", result.pipelineDecision);
}

main().catch(console.error);
```

Run the Final Project

```
npx tsx src/final-project/pipeline.ts
```

Skills Demonstrated in This Project

Phase 2 Skill	Where It's Used
`Send` (parallel fan-out)	`batchRouter` dispatches all articles simultaneously
`addConditionalEdges`	After `classifyTone`, routes positive vs. sensitive articles
Cycles / loops	`reviseArticle` → `checkRevision` → `reviseArticle` loop for sensitive articles
`recursionLimit`	Set to 150 to accommodate parallel × loop combinations
`Command`	`classifyTone` and `generateReport` use Command to update state and route together
Append reducer	`processedArticles` collects results from all parallel branches
Map-Reduce	`batchRouter` (map) + `generateReport` (reduce/aggregate)

Quick Reference Cheat Sheet

Conditional Edges

```
// Basic
graph.addConditionalEdges("nodeA", (state) => {
  return state.approved ? "nodeB" : "nodeC";
});

// With path map
graph.addConditionalEdges("nodeA", (state) => {
  return state.score > 80 ? "pass" : "fail";
}, { pass: "approveNode", fail: END });
```

```
// Set recursion limit
await compiled.invoke(input, { recursionLimit: 50 });
```

Loops

```
function routeFn(state): string {
  if (state.approved) return END;          // success exit
  if (state.iterations >= 3) return END;   // safety exit
  return "generateNode";                   // loop back
}
```

Command

```
import { Command } from "@langchain/langgraph";

function myNode(state): Command {
  return new Command({
    update: { field: "value" },   // update state
    goto: "nextNode"              // route to next node
  });
}
```

Send (Parallel Fan-Out)

```
import { Send } from "@langchain/langgraph";

function mapRouter(state): Send[] {
  return state.items.map(item =>
    new Send("processItem", { item, results: [] })
  );
}

// State must use append reducer for collected results
results: Annotation<string[]>({
  reducer: (existing, incoming) => [...existing, ...incoming],
  default: () => []
})
```

Map-Reduce Pattern

```
START → mapRouter (Send × N) → workNode (× N parallel) → aggregateNode → END
                                                          ^
                                                  append reducer collects N results
```

Decision Framework

I need to...	Use...
Route to different nodes based on state	`addConditionalEdges`
Route AND update state in one step	`Command`
Process a list of items in parallel	`Send`
Aggregate parallel results	Append reducer + reduce node
Prevent infinite loops	`recursionLimit` in `invoke()` config
Build a retry/correction loop	Cycle + iteration counter in routing function

---

End of Phase 2 Guide — Control Flow

Next: Phase 3 covers Persistence & Memory — checkpointers, state management, time travel, and the Store API.